

WebAssembly: A Low-Level Language that Works Well with JavaScript

WebAssembly (WA or Wasm) is a binary instruction format for a stack based virtual machine. It is designed as a portable target for the compilation of high-level languages like C, C++ and Rust, enabling deployment on the Web for client and server applications.



WebAssembly or Wasm is a technology that makes it possible to run high-performing, low-level code in the browser. In simple terms, Wasm is a programming language format (Abstract Syntax Tree) and execution environment (portable stack machine) that can be a compilation target for other programming languages. It can be embedded into the Web browser to provide a common way to support other languages, as an alternative to JavaScript.

WebAssembly goals

WebAssembly is being created as an open standard inside the W3C WebAssembly Community Group with the following goals.

- *To be fast, efficient and portable:* WebAssembly code can be executed at near-native speed across different platforms by taking advantage of common hardware capabilities.
- *To be readable and debuggable:* Wasm is a low-level assembly language, but it does have a human-readable text format (the specifications for which are still being finalised) that allows code to be written, viewed and debugged by hand.
- *To stay secure:* Wasm is designed to be run in a safe, sandboxed execution environment. Like other Web

code, it will enforce the browser's same origin and permissions policies.

- *Not to break the Web:* Wasm is designed so that it works well with other Web technologies and maintains backward compatibility.

How WebAssembly works

Here's a simple overview of what WebAssembly actually does.

In browsers today, JavaScript (JS) is executed in a virtual machine (VM), which optimises your code and squeezes out every ounce of performance. As things stand, JS is one of the fastest dynamic languages around. Yet, even though it is fast, it still can't compete with raw C and C++ code. This is where Wasm comes in.

WebAssembly runs in the same JavaScript VM, but performs much better. The two can communicate freely and don't exclude each other. This way you can have the best of both worlds — JavaScript's huge ecosystem and friendly syntax, combined with the near-native performance of WebAssembly.

Most people write Wasm modules in C and compile them to .wasm files, which aren't recognised by the browser directly; so they need something called JavaScript glue code to be loaded.